



Programação Orientada a Objetos

Herança

Maurício da Motta Braga
mmb2@cesar.school





*"Software entities
(classes, modules,
functions, etc.) should
be open for extension
but closed for
modification."*

Bertrand Meyer.

P00 – quatro pilares

Abstração

Encapsulamento

Herança

Polimorfismo



P00 – quatro pilares

Abstração

Encapsulamento

Herança

Polimorfismo



Herança

Mecanismo que permite reutilizar o código de classes existentes.

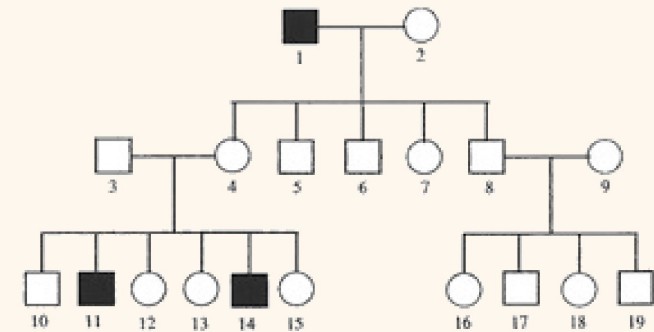
Herança introduz os conceitos de Superclasse e Subclasse, onde tudo que a superclasse tem a subclasse também possui.

Apenas novos atributos ou métodos precisam ser definidos nas novas classes.

Herança - Conceitos básicos

`class B extends A`

- ▶ Determinada pela palavra reservada **extends** após a definição da classe
- ▶ Superclasse = A -> classe “mãe”
- ▶ Subclasse = B -> classe “filha”
- ▶ A classe filha “herda” todos os atributos e métodos da classe mãe, **exceto construtores** – cada tipo tem o seu próprio construtor!
- ▶ A classe filha “herda” o tipo da classe mãe
- ▶ A classe filha “herda” a obrigação de chamar o construtor da classe mãe.



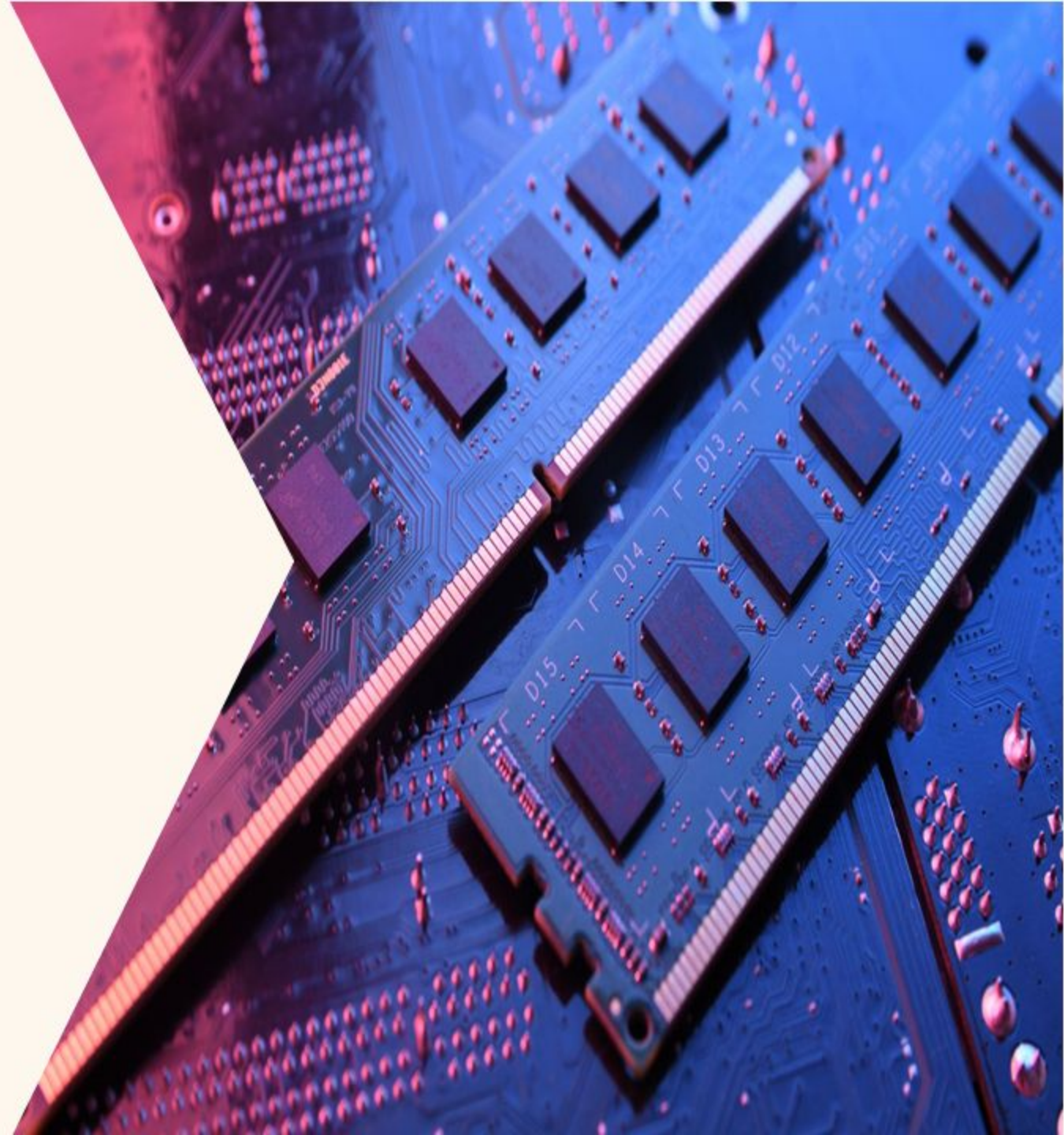
Herança e construtores

- ▶ A chamada ao construtor da classe mãe é feita através da palavra reservada “**super**”, seguida dos parâmetros a serem passados.
- ▶ A chamada ao construtor da superclasse deve ser a primeira linha de código de um construtor
- ▶ Se uma chamada ao construtor da superclasse não for adicionada, o JAVA coloca, em tempo de compilação, uma chamada implícita a um SUPOSTO construtor vazio da superclasse, ou seja, “**super()**”
- ▶ Há erros de compilação quando a chamada de um construtor a outro da superclasse, for incompatível com o construtor definido nesta.



Exemplo

- Imagine um programa para gerenciar a folha de pagamento de uma empresa. Tal programa foi criado com uma classe Empregado.
- A classe empregado foi definida para o empregado comum, que possui um salário fixo.
- O código da classe empregado pode ser visto no slide a seguir.

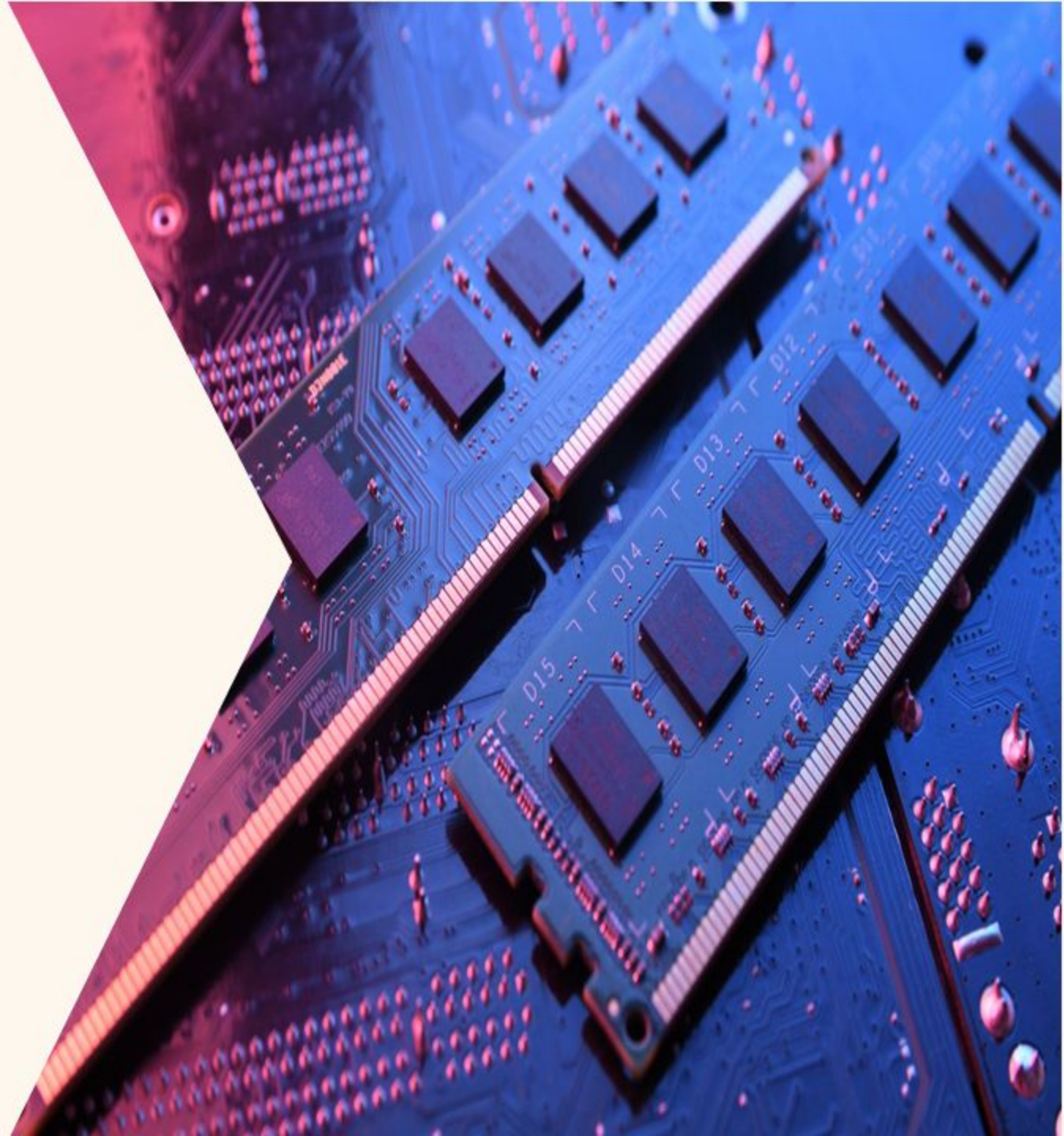


Exemplo

```
class Empregado {  
    private double salario;  
    private String nome;  
  
    public Empregado(String nome_emp, double val) {  
        nome = nome_emp;  
        salario = val;  
    }  
    public String getNome() { return nome; }  
    public double getSalario() { return salario; }  
    public double getPagamento() { return salario; }  
    // métodos set omitidos para simplificar.  
}
```

Exemplo

- Posteriormente, a empresa decidiu que deseja implementar também suporte a empregados que possuem além do salário fixo, uma comissão sobre as vendas realizadas. Essa é a única diferença entre os dois tipos de funcionários.
- Como implementar no programa suporte a esse funcionário comissionado?



Solução (?) utilizando encapsulamento

- ▶ Copiar todo o código da classe Empregado para uma nova classe EmpregadoComComissao, modificando a mesma para adicionar o suporte a comissões, que vão incrementar o salário dele.
 - ▶ Problemas desta solução:
 - ▶ Necessidade de manter dois códigos: Sempre que for necessário adicionar/corriger a classe Empregado, será necessário corrigir a classe EmpregadoComComissao também.
 - ▶ A melhor solução é utilizar herança.
-

Exemplo

```
class EmpregadoComComissao extends Empregado {  
    private double comissao;  
    public EmpregadoComComissao(String n_emp, double val) {  
        super(n_emp, val);  
        comissao = 0;  
    }  
    public void adicionarComissao(double percent, double valor) {  
        comissao = comissao + percent * valor;  
    }  
    @Override  
    public double getPagamento() {  
        return getSalario() + comissao;  
    }  
}
```

Solução utilizando herança

- ▶ EmpregadoComComissão herda de Empregado, obtendo todos os métodos e atributos definidos nesta.
 - ▶ Uma vez que um objeto do tipo EmpregadoComComissão é também um objeto do tipo Empregado, ele pode receber quaisquer mensagens que seriam enviadas a um objeto do tipo Empregado.
-

Exemplo

```
public static void main(String[] args) throws Exception {  
  
    EmpregadoComComissao c = new EmpregadoComComissao(  
        "Ricardo Silva", 3500);  
  
    c.adicionarComissao(0.05, 10000);  
    System.out.println("Nome: " + c.getNome());  
    System.out.println("Salário Fixo: " + c.getSalario());  
    System.out.println("Salário total: " + c.getPagamento());  
  
}
```

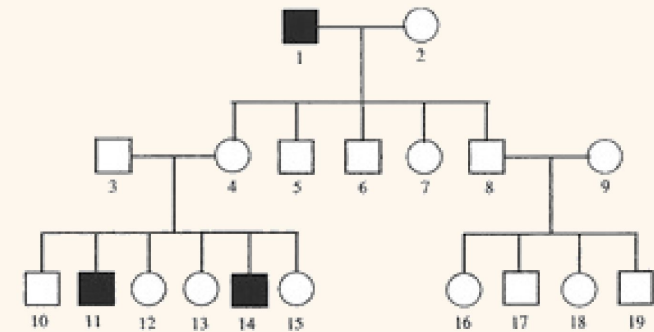
Porque utilizar herança?

Possibilidade de alterar o funcionamento herdado da classe mãe, criando uma classe mais adequada a um tipo de situação, sem precisar mexer na classe original.

Possibilidade de criar classes mais específicas, reutilizando código de classes genéricas.

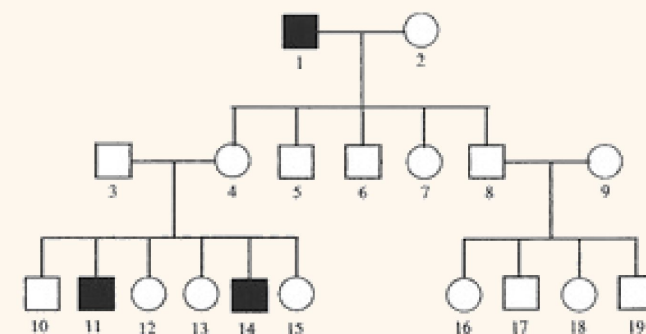
Herança

- ▶ Em Java, uma classe **só pode ter uma única classe mãe DIRETA**, mas pode ter “n” classes “avós, bisavós”, etc. (subindo na hierarquia)
 - ▶ **Não existe herança múltipla em JAVA**
- ▶ Uma classe pode ter N classes “filhas, netas, bisnetas”, etc. nos sentidos horizontal e descendo na hierarquia



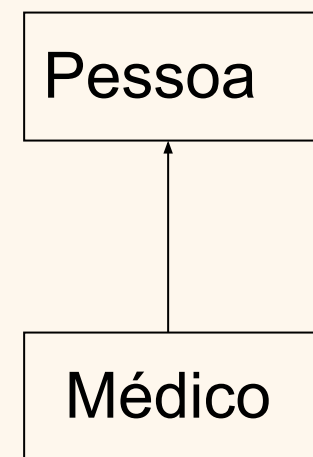
Herança - Conceitos básicos

- ▶ Objetos instanciados a partir de subclasses possuem todos os atributos e métodos das suas superclasses direta e indiretas
- ▶ A referência a atributos e métodos das superclasses em uma subclasse pode ser feita diretamente pelo nome do atributo/método ou pela palavra reservada `super`. Seguido do nome do atributo/método.



Validação da herança

- ▶ A forma de validar herança é montar a frase “é um” com as classes envolvidas.
- ▶ Exemplo: Médico e Pessoa.
- ▶ Todo médico é uma pessoa. (OK!)
- ▶ Toda pessoa é um médico (?)

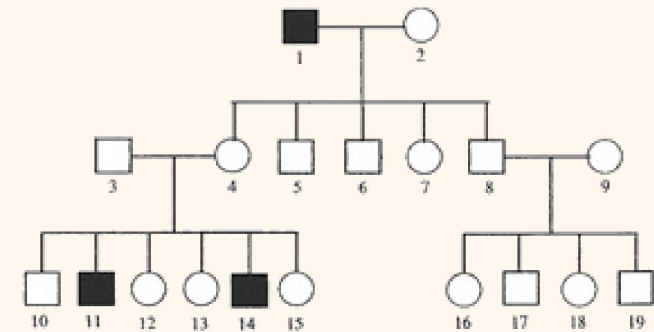


Validação da herança

- ▶ Validar a herança implica em verificar se faz sentido tratar a classe filha da mesma forma que se trata a classe mãe.
 - ▶ Uma classe filha pode apenas adicionar funcionalidade em relação a classe mãe. Remover funcionalidade não é permitido.
 - ▶ Caso haja necessidade de remover funcionalidade herdada da classe mãe, a hierarquia de classes deve ser revista.
-

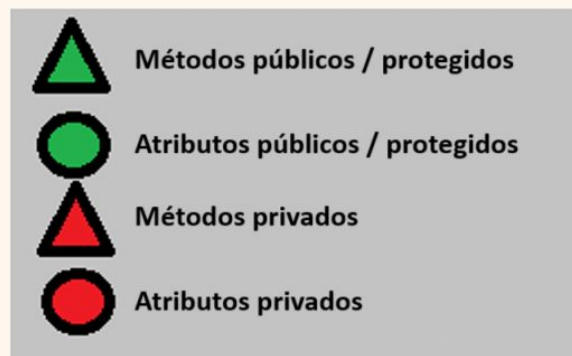
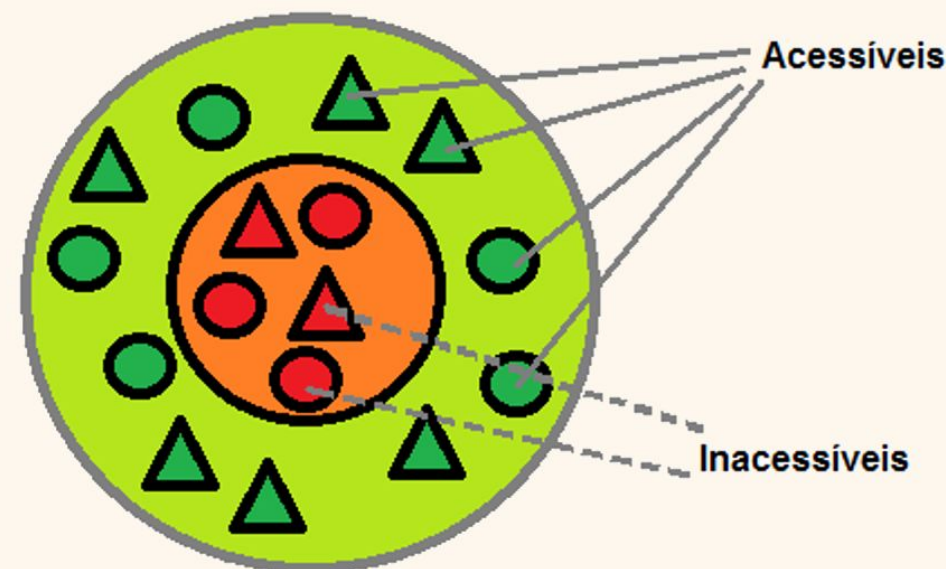
A classe Object

- ▶ Classe mãe de todas as classes no Java.
- ▶ Se uma classe não tiver superclasse explicitamente declarada, ela estende automaticamente de Object.
- ▶ Toda classe herda, direta ou indiretamente, de Object.



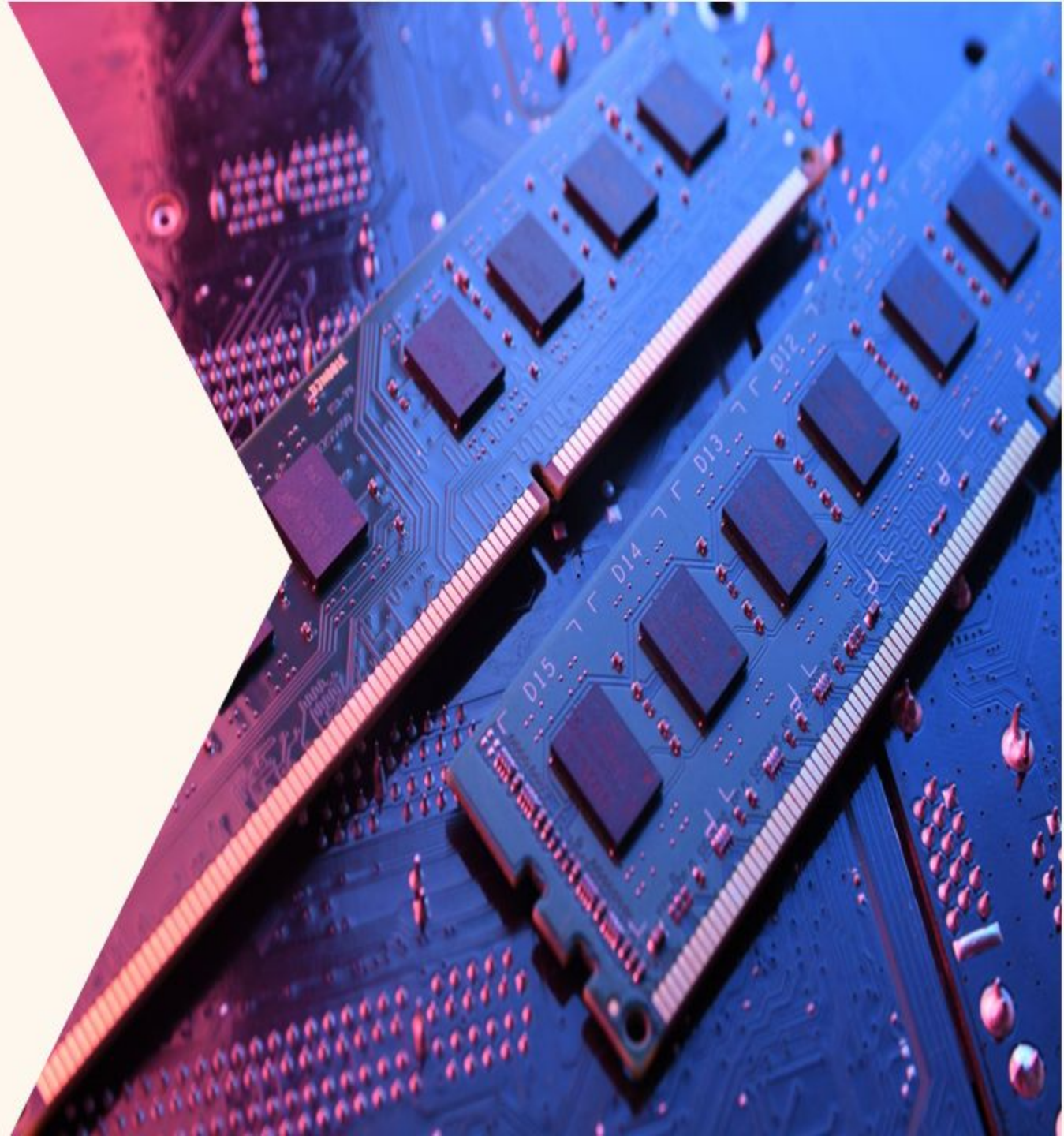
Herança e modificadores de acesso

- ▶ Subclasses não acessam atributos e métodos privados de suas superclasses.
- ▶ Subclasses só acessam atributos e métodos default das suas superclasses se estas estiverem no mesmo pacote da subclasse.
- ▶ Subclasses só acessam atributos e métodos protected ou public das suas superclasses.



Prática

- Imagine um programa para gerenciar uma grande loja com produtos variados.
- Tal programa irá conter uma classe Produto, definida para um produto genérico.
- Atributos do produto: código, nome, preço e desconto_a_vista.

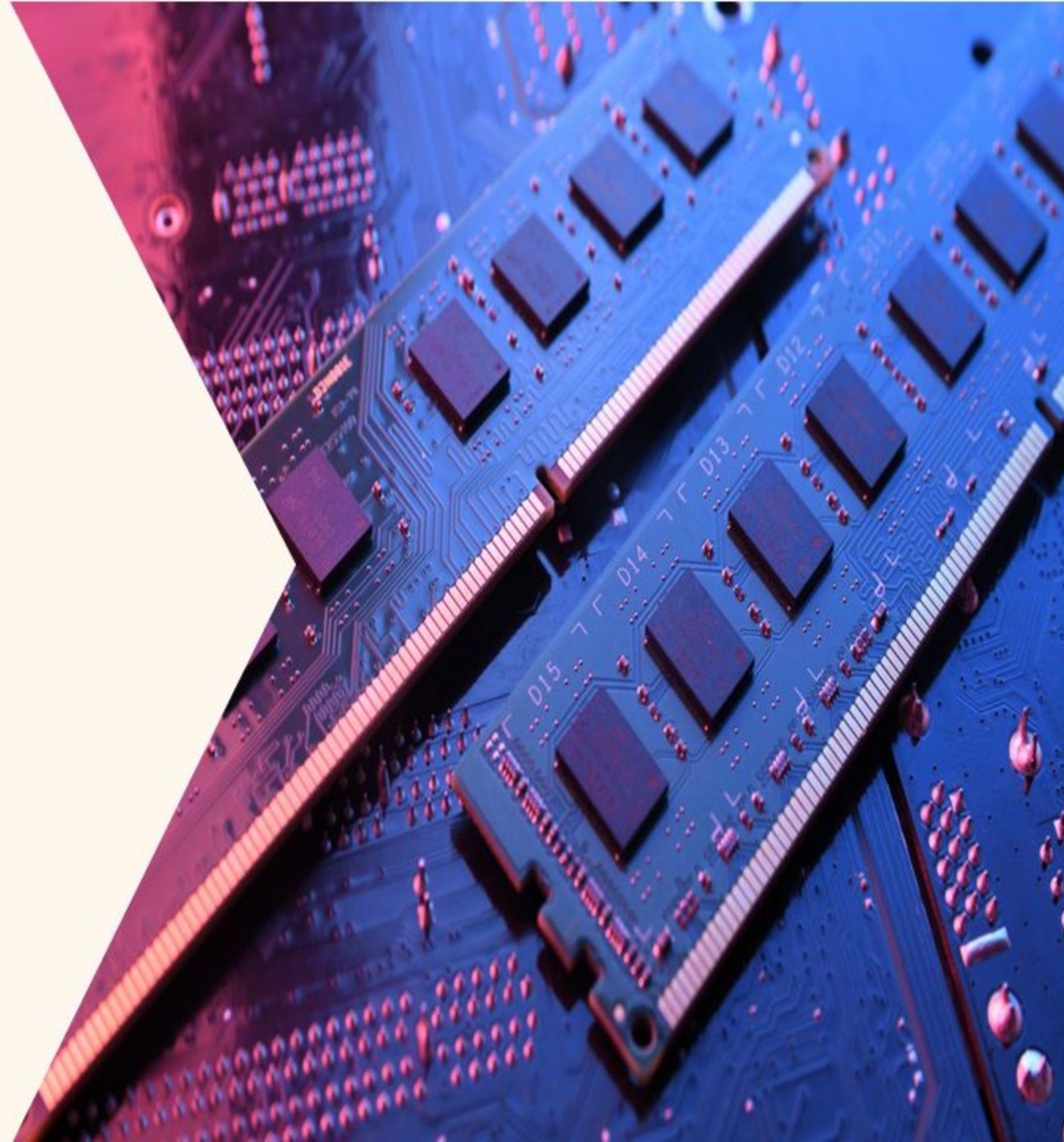


Exemplo

```
public class Produto {  
    private int codigo;  
    private String nome;  
    private double preco;  
    private double desconto;  
    public Produto(int codigo, String nome, double preco, double desc) {  
        super();  
        this.codigo = codigo;  
        this.nome = nome;  
        this.preco = preco;  
        this.desconto = desc;  
    }  
    public String toString() {  
        return "Eu sou um produto de código " + getCodigo() + ", nome = "+getNome() + ", preço = " +getPreco() +  
            " e desconto para pagamento a vista = "+getDesconto() + " %";  
    }  
    // Métodos get e set omitidos  
}
```

Prática

- Crie uma subclasse de Produto, a classe Livro.
- Essa classe deverá, assim como Produto, conter um construtor, métodos de acesso e um método toString() que retorna uma string que informa o nome da classe e o valor de seus atributos (tanto os da classe Livro como da classe Produto).
- Atributos do Livro: autor e editora;

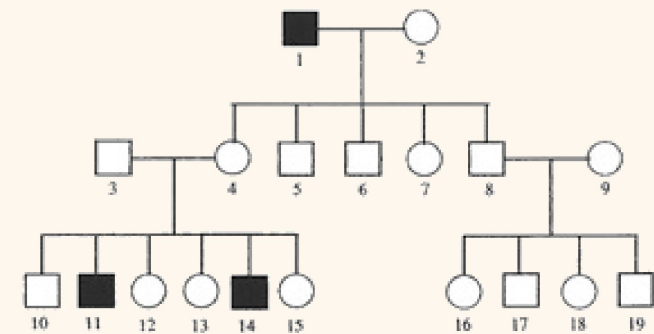


Solução

```
class Livro extends Produto {
    private String autor;
    private String editora;
    public Livro(int codigo, String nome, double preco, double desc, String autor, String
editora) {
        super(codigo, nome, preco, desc);
        this.autor = autor;
        this.editora = editora;
    }
    // métodos de acesso omitidos
    public String toString() {
        return "Eu sou um livro de código " + getCodigo() + ", nome = "+getNome() + ",
preço = " +getPreco() + " , desconto para pagamento a vista = "+getDesconto() + " %" +
" autor = "+autor + " e editora = " + editora;
    }
}
```

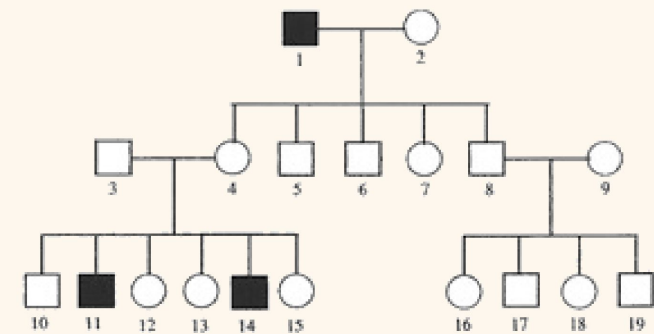
Métodos da classe Object

- ▶ Entre os métodos definidos nessa classe, temos 3 métodos que são importantes para fazer comparações de objetos bem como debugging: **equals**, **hashCode** e **toString**.
- ▶ A utilização adequada desses métodos adiciona recursos valiosos para utilização dos objetos em um software.
- ▶ Esses métodos costumam ser redefinidos nas classes novas, para melhor servir a elas.



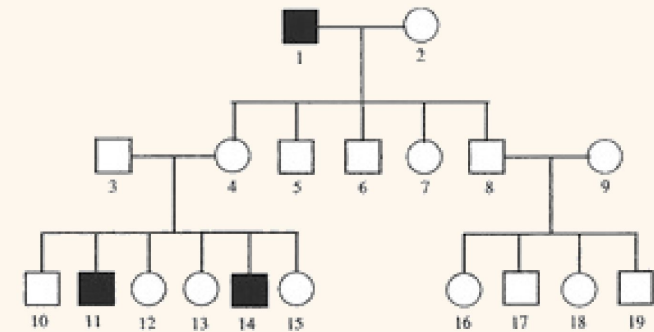
Método hashCode

- ▶ **int hashCode()** – Tem como objetivo retornar um valor inteiro, gerado por um algoritmo de hash, que é usado para identificar exclusivamente objetos em coleções baseadas em hash, como HashMap, HashSet e Hashtable.
- ▶ Objetivo é retornar um valor que sirva como identificador único para o objeto em questão.



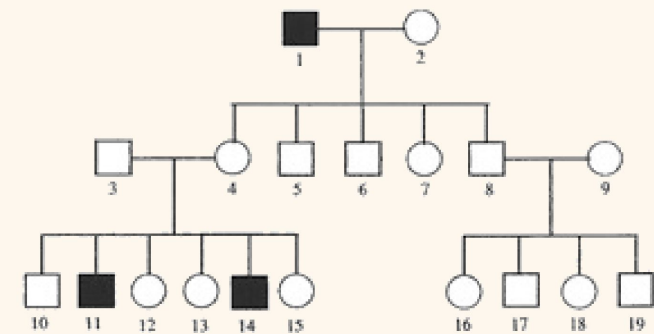
Método equals

- ▶ **boolean equals(Object o)** – Tem como objetivo comparar o objeto atual com o recebido como parâmetro para identificar se eles são iguais.
- ▶ A implementação existente em Object compara o endereço de memória para ver se são o mesmo objeto na memória.
- ▶ Ao sobrescrever o método equals(), deve-se também substituir o método hashCode() para garantir que objetos iguais produzam o mesmo código hash.



Método toString

- ▶ **String toString()** – retorna uma string cujo objetivo é “representar” o objeto em questão.
- ▶ Na implementação em Object, retorna a identificação do tipo do objeto seguido por um caractere “@” e o seu código hash.



Benefícios do uso da herança

Incentiva a criação de famílias de classes com reuso de código.

Flexibilidade: Poder criar novos subtipos de uma classe e utilizá-los aonde a superclasse é utilizada sem precisar recompilar o código.

Facilidade maior para adicionar novas funcionalidades.



**Slides adaptados a partir
do material produzido pelo
prof. Eduardo Calábria**

`egc2@cesar.school`





**MAURÍCIO DA
MOTTA BRAGA**



mmb2@cesar.school

